中国石油大学(华东)
CHINA UNIVERSITY OF PETROLEUM

本 科 毕 业 设 计（论文）

题　目：线性表的设计和实现

——基于 C++ Builder 6.0 的实现

姓　　名：_____

学　　号：_____

学　　院：_____

专　　业：_____

指导教师：_____

年　　月　　日

学位论文原创性声明

本人所提交的学位论文《线性表的设计和实现》，是在导师的指导下，独立进行研究工作所取得的原创性成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中标明。

本声明的法律后果由本人承担。

论文作者（签名）：　　指导教师确认（签名）：

年　月　日　　年　月　日

学位论文版权使用授权书

本学位论文作者完全了解中国石油大学（华东）有权保留并向国家有关部门或机构送交学位论文的复印件和磁盘，允许论文被查阅和借阅。本人授权中国石油大学（华东）可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其它复制手段保存、汇编学位论文。

论文作者（签名）：　　指导教师（签名）：

年　月　日　　年　月　日

线性表的设计和实现

—— 基于 C++ Builder 6.0 的实现

摘　要

数据结构算法设计和演示（C++）树和查找是在面向对象思想和技术的指导下，采用面向对象的编程语言（C++）和面向对象的编程工具（Borland C++ Builder 6.0）开发出来的小型应用程序。它的功能主要是将数据结构中链表、栈、队列、树、查找、图和排序部分的典型算法和数据结构用面向对象的方法封装成类，并通过类的对外接口和对象之间的消息传递来实现这些算法，同时利用 C++ Builder 6.0 中丰富的控件资源和系统资源对算法实现过程的流程和特性加以动态的演示，从而起到在数据结构教学中帮助理解、辅助教学和自我学习的作用。

关键词：数据结构；面向对象；可视化；算法

The design and implementation of the linear form
—— Based on C++ Builder 6.0

Abstract

This paper introduces a small application program developed based on object-oriented thinking and technology, using C++ programming language and Borland C++ Builder 6.0 development tool. It encapsulates typical algorithms and data structures of linear lists, stacks, queues, trees, searching, graphs and sorting in data structure into classes by object-oriented method. The algorithms are realized through class interfaces and message passing between objects. Meanwhile, it dynamically demonstrates the process and characteristics of algorithm implementation with rich controls and system resources, which helps students understand data structure knowledge and assists teaching and self-learning.

Keywords: data structure; object-oriented; visualization; algorithm

目　录

第 1 章 引言 1
第 2 章 线性表的基本理论知识 2
2.1 线性表的定义 2
2.2 线性顺序表 2
2.2.1 三级标题名 2
2.2.2 三级标题名 2
2.3 线性链表 2
第 3 章 设计的主体内容 3
3.1 系统结构的设计 3
3.2 交互界面的设计和实现 3
3.3 线性表的 OOP 设计 3
3.3.1 线性表的顺序存储的实现 4
3.3.2 线性表的链表存储的实现 4
第 4 章 结果分析与讨论 5
第 5 章 结论 7
致　谢 8
参考文献 9
附　录 10
附录 A 名词术语及缩略词 10
附录 B 线性表 Python 实现 11

第 1 章 引言

计算机与网络技术的高速发展，特别是面向对象技术的出现，使得 C++ 的软件开发得到了迅速普及。

本课题主要研究线性表的两种基本存储结构（顺序存储和链式存储）的设计与实现，采用面向对象的方法将线性表的操作封装为类，实现插入、删除、查找、遍历等核心功能，并开发可视化交互界面，直观展示算法执行过程。

正文继续撰写时，应保证总字数不少于学校要求，全文的单位、符号和缩写应尽量遵循国际单位制 SI 及相关学术写作规范。

第 2 章 线性表的基本理论知识

2.1 线性表的定义

线性表是最简单、最常用的一种数据结构。线性表[1] 是 n（$n \geq 0$）个数据元素的有限序列。

2.2 线性顺序表

线性表的顺序存储结构的特点是为表中相邻的元素 a_i 和 a_{i+1} 赋以相邻的存储位置。

2.2.1 三级标题名

2.2.2 三级标题名

（1）三级以下标题

2.3 线性链表

线性表的链式存储结构的特点是用一组任意的存储单元存储线性表的数据元素，这组单元可以是连续的，也可以是不连续的，因此能够更加灵活地处理动态数据结构。

第 3 章 设计的主体内容

在着手进行上机设计之前首先做好大量准备：应熟悉课题，进行调查研究，收集国内、外资料并进行分析研究；同时还应完成交互界面的总体构思与实现方案设计。

3.1 系统结构的设计

3.2 交互界面的设计和实现

交互界面的设计应遵循简洁、直观、易用的原则，方便用户操作和观察算法执行过程。

$$i(t) = I_{\mathrm{m}} e^{-\alpha t} \sin(\omega t + \varphi) \tag{3-1}$$

由公式（3-1）可以看出，电流随时间呈指数衰减的正弦变化规律。

3.3 线性表的 OOP 设计

计算机内部可以采用两种不同方法来表示一个线性表，它们分别是顺序表示法和链表表示法。

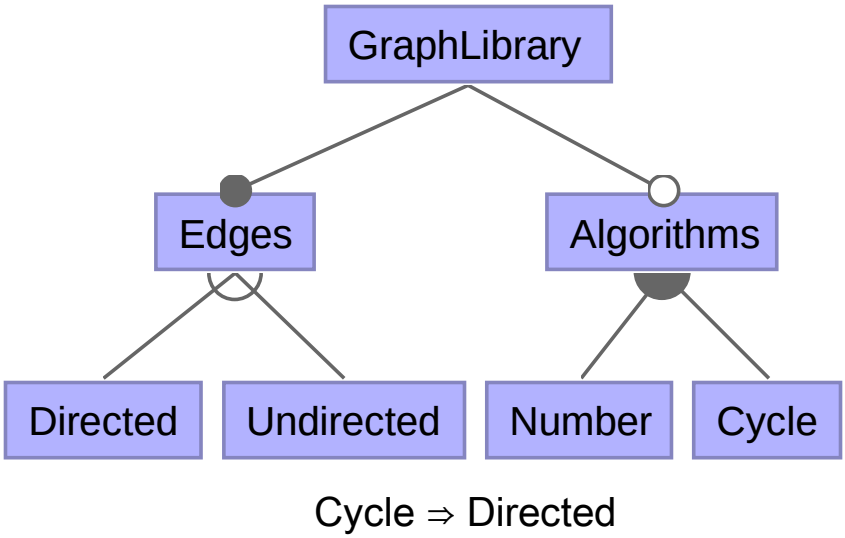

图 3-1　过阻尼响应

过阻尼响应如图 3-1 所示，此时系统无振荡，输出缓慢趋近于稳态值。

3.3.1 线性表的顺序存储的实现

以上是顺序表的实现过程，第 1－16 行包含了 list 类的说明，接下来是成员函数的定义。

3.3.2 线性表的链表存储的实现

链表的实现包括两个类定义，第一个是 link 类，第二个是 list 类。由于一个链表由若干个单独的链结点对象组成，因此一个链结点应当作为单独的 link 类实现。

为使本模板在页眉页脚、图表编号和跨页排版方面能够与原始 Word 模板逐页对照，本章在后续页面中继续保留实现说明示例。实际撰写时，可继续补充系统流程、类图、测试步骤或核心代码设计说明，并根据篇幅需要增减内容。

第 4 章 结果分析与讨论

例如由于起初未能真正掌握各种控件的功能，我设想是要一个下拉菜单，但是学识肤浅的我试了很多种就是达不到我要的效果，经过查阅资料和向老师同学请教，最终解决了这个问题。

关于激光入射功率密度对导轨滚道表面硬化层深和显微硬度的影响如表 4-1 所示。为验证三线表在跨页场景下的排版效果，以下表格故意放置在页面底部附近，若内容超出当前页，应在第二页重复表头右上方显示“续表 4-1”。

表 4-1　激光入射功率密度对导轨滚道表面硬化层深和显微硬度的影响

试验编号	功率密度 $/W\cdot cm^{-2}$	辐照时间 /s	显微硬度 /HV	硬化层深 /mm
t-1	6.37×10^{3}	0.067	570，456	0.354
t-2	6.37×10^{3}	0.067	570，456	0.354
t-3	6.37×10^{3}	0.067	570，456	0.354
t-4	6.37×10^{3}	0.067	570，456	0.354

续表 4-1

试验编号	功率密度 /$W \cdot cm^{-2}$	辐照时间 /s	显微硬度 /HV	硬化层深 /mm
t-5	6.37×10^3	0.067	570，456	0.354
t-6	6.37×10^3	0.067	570，456	0.354
t-7	6.37×10^3	0.067	570，456	0.354
t-8	6.37×10^3	0.067	570，456	0.284
t-9	6.37×10^3	0.067	570，456	0.354
t-10	6.37×10^3	0.067	570，456	0.354

第 5 章 结论

本课题采用 C++ 语言、面向对象的设计方法实现数据结构中线性表的重要算法，完成了顺序表和链表的基本操作封装，并开发了可视化演示界面，能够直观展示算法的执行过程。

同时，本设计还存在着许多不足之处，如：仅实现了线性表的基本操作，未包含更复杂的应用场景；可视化界面的交互性有待进一步提升；未对算法的时间复杂度和空间复杂度进行详细的对比分析。

致　谢

大学四年的学习生活即将结束，在此，我要感谢所有曾经教导过我的老师和关心过我的同学，他们在我成长过程中给予了我很大的帮助。本文能够成功地完成，要特别感谢我的导师 XXX 教授的关怀和教导。

感谢实验室的师兄师姐在课题研究过程中给予的帮助和建议，感谢我的家人和朋友一直以来的支持和鼓励。

参考文献

[1] 严蔚敏，吴伟民．数据结构（C 语言版）[M]．北京：清华大学出版社，1997.

附　录

附录 A 名词术语及缩略词

OOP：Object-Oriented Programming，面向对象程序设计。GUI：Graphical User Interface，图形用户界面。

附录中的插图、表格和公式同样建议使用标准环境自动生成编号。示例中，系统整体架构如图 A1 所示，核心函数说明见表 A1，二次函数表达式见式 A1。

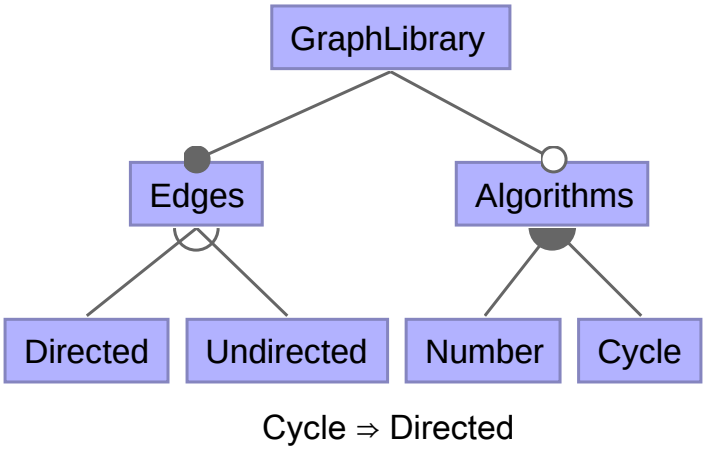

图 A1　系统整体架构图

表 A1　核心函数说明表

函数名	功能说明
InitList	初始化线性表
InsertElem	插入元素
DeleteElem	删除元素

$$f(x) = ax^2 + bx + c \tag{A1}$$

附录 B 线性表 Python 实现

以下给出一种基于 Python 的简易单链表实现，涵盖插入、删除与遍历操作，供读者参考。

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        cur = self.head
        while cur.next:
            cur = cur.next
        cur.next = new_node

    def delete(self, key):
        cur = self.head
        prev = None
        while cur and cur.data != key:
            prev = cur
        cur = cur.next
        if not cur:
            return
        if prev:
            prev.next = cur.next
        else:
            self.head = cur.next
```

```python
    def display(self):
        cur = self.head
        while cur:
            print(cur.data, end=" -> ")
            cur = cur.next
        print("None")

# 示例用法
lst = LinkedList()
for x in [1, 2, 3, 4]:
    lst.append(x)
lst.display()
```